EX25 · Force allocation with linear algebra

Hands-on engineering assignment · estimated 60–90 min · prerequisites: functions, NumPy, JSON and pytest.

LANGUAGE / ID	UNIT	HANDOUT
EN · EX25	algebra / vectors and linear systems	Exercise / Mission

1 · Mission goal

A platform has **two ideal actuators** which generate forces along fixed directions in a plane. Your program must calculate each actuator force in newtons to reach a target resultant, verify the solution and study its response to a small change in the target.

> **The challenge:** translate a physical model into a linear system, implement reusable functions, check units and magnitudes, and validate your code with 10 automated tests.

2 · Engineering model

The two actuator unit directions are the columns of matrix A. Vector u contains the force magnitudes; vector b is the desired resultant force.

$$Au = b$$

```
A = [[ 0.8,  0.6],
     [ 0.6, -0.8]]

b       = [14.00, 2.00]      # target Fx,Fy [N]
delta_b = [ 0.00, 0.01]      # variation [N]

# Solve for u = [u1, u2], in newtons.
```

Column one is actuator 1’s direction: (0.8, 0.6). Column two is actuator 2’s direction: (0.6, −0.8). Both direction vectors have unit norm. Forces are assumed to be concurrent; torque and saturation constraints are ignored.

3 · Deliverables and exact folder

```
Python_Engineering/
└── 02_numerical_methods/
    └── algebra/
        └── class_17_vectors_and_linear_systems/
            ├── main.py
            ├── inputs/actuators_case.json
            ├── src/algebra.py       # complete the TODOs
            ├── tests/test_algebra.py
            ├── docs/ES/ and docs/EN/
            └── outputs/
```

The supplied starter project includes code scaffolding and tests. **There is no implemented solution:** the NotImplementedError exceptions are intentional.

4 · Functions to implement

Function	Input	Output / check
resultant_force(A, u)	2×2 matrix; (2,) magnitudes	Vector A @ u, shape (2,)
solve_forces(A, b)	2×2 matrix; (2,) target	u; validate dimensions and finite values
residual_norm(A, u, b)	A, solution u, target b	Scalar residual norm, in N
relative_residual(A, u, b)	A, u and b	Dimensionless number; reject ||b|| = 0
sensitivity_analysis(A, b, delta_b)	A, target, perturbation	Dictionary with 7 documented keys

Step 1 · Vector and matrix calculations

- Implement **resultant_force** with the @ operator, not element-wise multiplication (*).
- Check **shape**, data types and the physical meaning of each component.

Step 2 · Solve the system

- Implement **solve_forces** using **np.linalg.solve**. Avoid explicitly calculating a matrix inverse.
- Raise **ValueError** for invalid shapes or NaN/Inf; allow **np.linalg.LinAlgError** to propagate for a singular matrix.

Step 3 · Quantify reconstruction error

- Compute absolute residual $||A @ u - b||_2$, in newtons.
- Divide by $||b||_2$ to obtain the dimensionless relative residual.

Step 4 · Assess sensitivity

- Solve twice: A @ u = b and A @ u_perturbed = b + delta_b.
- Compute delta_u, its norm, the relative input/output changes and **np.linalg.cond(A, 2)**.

5 · Expected validation results

Check	Reference result	Tolerance / unit
Original u	[12.4, 6.8]	10^{-10} N
A @ u	[14.0, 2.0]	10^{-10} N
Absolute / relative residual	Near zero	Both < 10^{-10}
Perturbed u	[12.406, 6.792]	10^{-10} N
delta_u	[0.006, −0.008]	10^{-10} N
$\|\|delta_u\|\|_2$	0.010000	N
Condition $\kappa_2(A)$	1.000000	Dimensionless

The relative changes in b and u are both about 0.0007071, or **0.07071%**. Explain why they coincide here.

6 · Testing and execution

```
# VS Code terminal, inside class_17_vectors_and_linear_systems
python -m pip install -r requirements.txt
python -m pytest -q
python main.py
```

The 10 tests cover direct calculation, correct solution, residuals, zero target, invalid dimensions, non-finite input, singular matrices, and sensitivity. **They initially fail** because the functions have not been implemented.

7 · Engineering questions

- What does each column of A represent? What are the units of u and b?
- Why does a small residual not always guarantee a reliable result?
- What does $\kappa_2(A) = 1$ mean for this actuator geometry?
- What physical constraints would be introduced by positive-only forces and maximum actuator capacity?

8 · Optional extension

Construct a second matrix with almost-parallel directions (e.g., rows [1, 1] and [1, 1.000001]). Solve a case before and after a small target perturbation and compare condition numbers. A mathematical solution may not always be physically feasible.

> **Completion standard:** 10 passing tests, successful execution of main.py, expected-value comparison, and a brief interpretation of residual and condition number.